

ĐÁNH GIÁ MÔ HÌNH

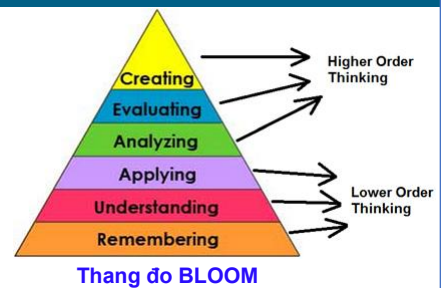


1



Mục tiêu

- Hiểu đánh giá mô hình
- Phân tích Over-fitting, Under-fitting và hiểu cách chọn mô hình
- Hiểu hồi quy Ridge $/rɪdʒ/$



2

2



Nội dung

1. Kỹ thuật đánh giá mô hình
2. Overfitting - Underfitting - Chọn mô hình
3. Hồi quy Ridge

3

3



Bộ dữ liệu

Lý thuyết và Thực hành:

<https://raw.githubusercontent.com/datasethub/ds105/master/Model-Evaluation-and-Refinement.csv>

4

4



1. Kỹ thuật đánh giá mô hình

- 1.1. Phương pháp đánh giá mô hình
- 1.2. Hiệu suất
- 1.3. Kiểm chứng chéo
- 1.4. Tóm tắt các thang đo đánh giá

5

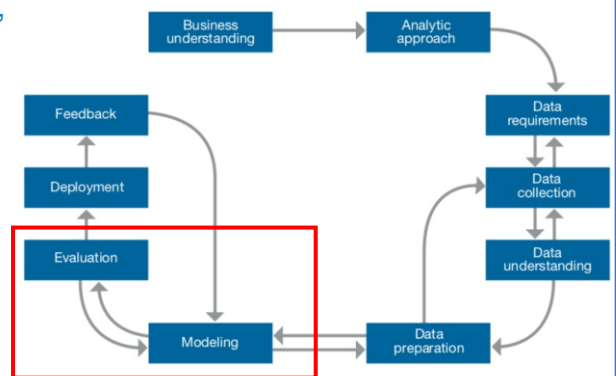
5



1.1. Phương pháp đánh giá mô hình

Giới thiệu

- Đánh giá tập mẫu cho phép chúng ta biết được mô hình có phù hợp với dữ liệu được dùng để phát triển mô hình.
- Vấn đề đặt ra:
 - Không biết được mô hình có thể dự đoán các giá trị mới tốt như thế nào.
- Giải pháp
 - Đánh giá tập dữ liệu dùng để phát triển mô hình (gọi là training set)
 - Tập dùng để kiểm tra (gọi là testing set)



John B. Rollins, Ph.D.

6

6



1.1. Phương pháp đánh giá mô hình

Training/Testing Set

- Bộ dữ liệu



Tách bộ dữ liệu thành:

- Training set (70%):
- Testing set (30%):
- Phát triển mô hình với tập huấn luyện (**training set**)
- Dùng tập kiểm thử (**testing set**) để đánh giá hiệu suất dự đoán của mô hình.
- Khi hoàn thành đánh giá mô hình nên sử dụng lại tất cả dữ liệu để phát triển lại mô hình để có hiệu suất tốt nhất.

...

7

7



1.1. Phương pháp đánh giá mô hình

Hàm `train_test_split()`

- Tách bộ dữ liệu thành hai bộ train và test bằng kỹ thuật ngẫu nhiên

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.3, random_state=0)
```

- **x_data**: biến độc lập
- **y_data**: biến phụ thuộc => df['price']
- **x_train, y_train**: biến độc lập và phụ thuộc của bộ train
- **x_test, y_test**: biến độc lập và phụ thuộc của bộ test
- **test_size**: kích cỡ bộ test, đơn vị % (30%)
- **random_state**: số dùng để làm cơ sở kiểm soát việc xáo trộn dữ liệu

...

8

8



1.2. Hiệu suất

Generalization performance (Hiệu suất tổng thể, hay Hiệu suất chung)

- Là đánh giá khả năng mô hình hoạt động tốt trên dữ liệu mới.
- Mô hình dự đoán chính xác phần lớn các trường hợp của **bộ test** thì mô hình đó đạt được độ khái quát hóa tốt (tức là *generalization performance*).
- **Lỗi tổng thể** (*generalization error*) là thang đo thể hiện độ tốt của **bộ train** dùng để phát triển mô hình.
- Đo lường sự khác biệt giữa hiệu suất của mô hình trên **bộ train** và **bộ test**.
- Lỗi tổng thể được tính bằng cách sử dụng **bộ test** đưa vào mô hình.

...

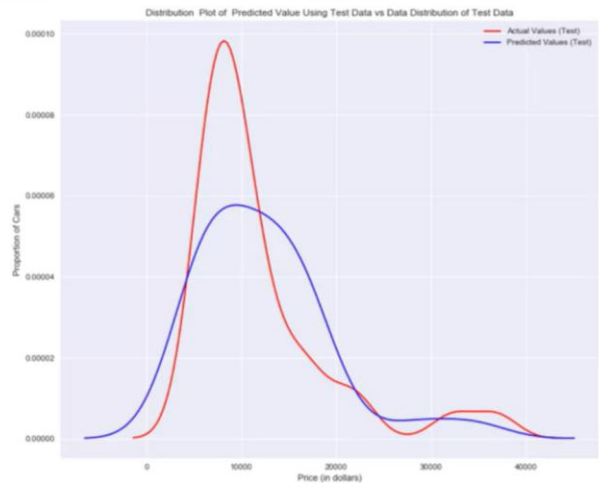
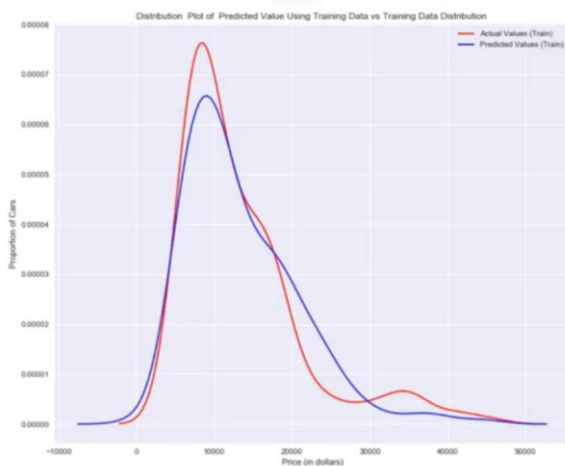
9

9



1.2. Hiệu suất

Generalization error



...

10

10



1.3. Kiểm chứng chéo

Cross validation (*k*-Fold Cross-Validation)

- Cross validation (kiểm chứng chéo) là phương pháp thống kê lấy mẫu để kiểm tra đánh giá độ chính xác mô hình.
- So sánh và chọn ra mô hình tốt nhất.
- Sử dụng phương pháp Cross-Validation với *k*-Fold
=> tương đương: **k-Fold Cross-Validation**

...

11

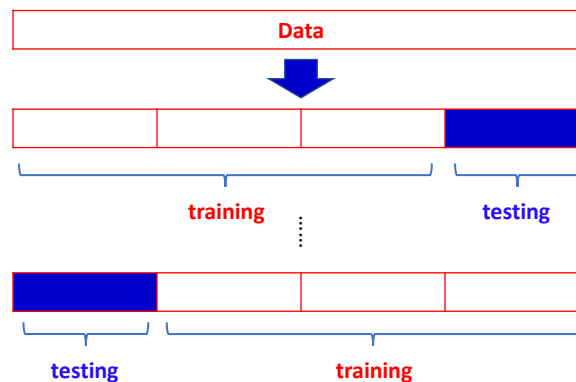
11



1.3. Kiểm chứng chéo

Cross validation (*k*-Fold Cross-Validation)

- Số liệu đánh giá ngoài mẫu phổ biến nhất.
- Sử dụng dữ liệu hiệu quả hơn (dữ liệu được sử dụng cho cả train và test)



...

12

12



1.3. Kiểm chứng chéo

Cross validation (*k*-Fold Cross-Validation)

Fold 1 Fold 2 Fold 3 Fold 4 Fold 5

Train	Validation	Kết quả
Fold 1 Fold 2 Fold 3 Fold 4	Fold 5	a1
Fold 1 Fold 2 Fold 3 Fold 5	Fold 4	a2
Fold 1 Fold 2 Fold 5 Fold 4	Fold 3	a3
Fold 1 Fold 5 Fold 3 Fold 4	Fold 2	a4
Fold 5 Fold 2 Fold 3 Fold 4	Fold 1	a5

...

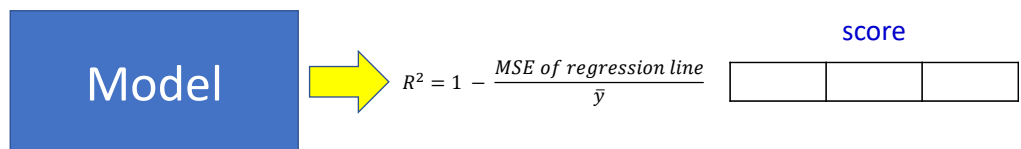
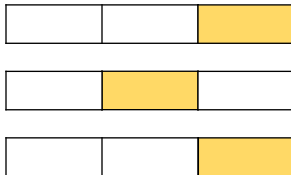
13

13



1.3. Kiểm chứng chéo

Hàm `cross_val_score()`



...

14

14



1.3. Kiểm chứng chéo

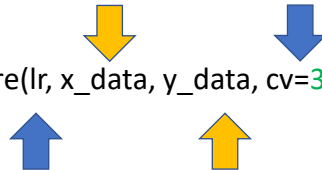
Hàm `cross_val_score()`

```
from sklearn.model_selection import cross_val_score
```

['horsepower']

```
scores = cross_val_score(lr, x_data, y_data, cv=3)
```

```
np.mean(scores)
```



...

15

15



1.3. Kiểm chứng chéo

Hàm `cross_val_predict()`

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
-------	-------	-------	-------	-------	-------	-------	-------	-------

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
-------	-------	-------	-------	-------	-------	-------	-------	-------

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
-------	-------	-------	-------	-------	-------	-------	-------	-------

Model

...

16

16



1.3. Kiểm chứng chéo

Hàm `cross_val_predict()`

- Trả về giá trị dự đoán của mỗi phần tử trong bộ test
- Có interface (giao diện) sử dụng giống như `cross_val_score()`

```
from sklearn.model_selection import cross_val_predict
```

```
yhat= cross_val_predict (lr2e, x_data, y_data, cv=3)
```

...

17

17



1.4. Các thang đo đánh giá

Tóm tắt các thang đo đánh giá

- In-Sample Evaluation:
 - **Mean Squared Error (MSE)**: gọi `mean_squared_error()` trong `metrics`
 - **R-squared (R^2)**: gọi `score()` trong mô hình.
- Out-Sample Evaluation:
 - **Cross validation (CV)**: gọi `cross_val_score()` trong `model_section` để thẩm định chéo mô hình hồi quy.

- Tìm hiểu **Root Mean Squared Error (RMSE)** $RMSE = \sqrt{\frac{\sum_{i=1}^n (p_i - a_i)^2}{n}}$

...

18

18



1.4. Các thang đo đánh giá

Tóm tắt các thang đo đánh giá

- Ngoài ra, còn các thang đo như:

- Relative Squared Error (RSE)

$$RSE = \frac{\sum_{i=1}^n (p_i - a_i)^2}{\sum_{i=1}^n (\bar{a} - a_i)^2}$$

- Mean Absolute Error (MAE)

$$MAE = \frac{\sum_{i=1}^n |p_i - a_i|}{n}$$

- Relative Absolute Error (RAE)

$$RAE = \frac{\sum_{i=1}^n |p_i - a_i|}{\sum_{i=1}^n |\bar{a} - a_i|}$$

=> Nhiệm vụ: tìm hiểu thêm các thang đo đã liệt kê bên trên.

...

19

19



2. Overfitting - Underfitting – Chọn mô hình

- 2.1. Overfitting (mô hình quá khớp)
- 2.2. Underfitting (mô hình chưa khớp)
- 2.3. Chọn mô hình
- 2.4. Minh họa



...

20

20



2.1. Overfitting

Overfitting (Mô hình quá khớp)

- Là mô hình áp dụng đạt kết quả **cao (tốt)** cho bộ dữ liệu **train** nhưng áp dụng đạt kết quả **thấp (không tốt)** cho bộ dữ liệu **test**.

Nguyên nhân:

- Do dữ liệu nhiễu, hoặc dữ liệu bất thường (**outlier**) được chọn góp phần đưa ra quy luật mô hình.
- Hay, dữ liệu nhiễu/bất thường tham gia train góp phần “**bẻ gãy**” đường hồi quy “=> **làm mềm mại đường hồi quy**”

...

21

21



2.2. Underfitting

Underfitting (Mô hình chưa khớp)

- Là mô hình chưa có độ chính xác cao đối với tập dữ liệu test sau khi phát triển.

Nguyên nhân và cách khắc phục:

- Do tập dữ liệu train chưa đạt chuẩn (cách kiểm tra: thực hiện phân tích thăm dò lại bộ dữ liệu)
- Bổ sung thêm dữ liệu train.
- Hoặc đổi thuật toán.

...

22

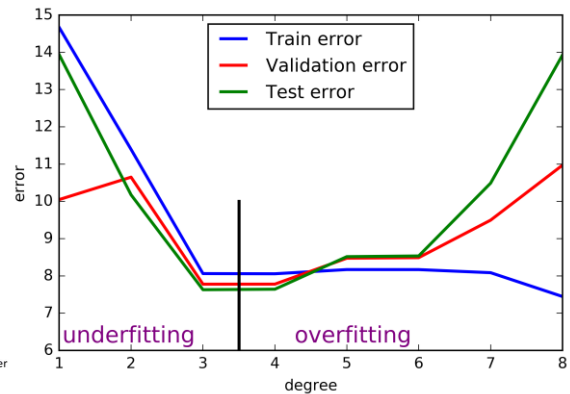
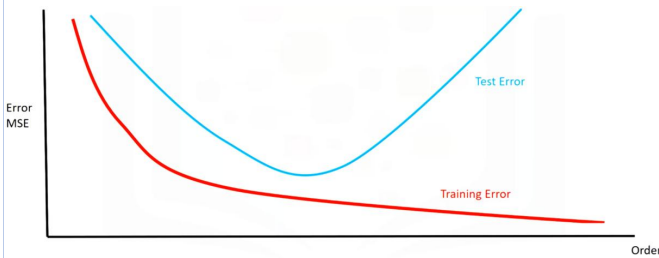
22



2.3. Chọn mô hình

Model selection => Good Fitting

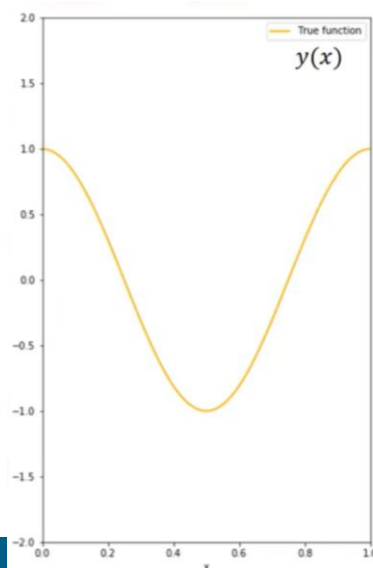
- Chọn mô hình làm sao để vừa khớp (Good Fitting).
- Nằm giữa Overfitting và Underfitting.



23



2.3. Chọn mô hình

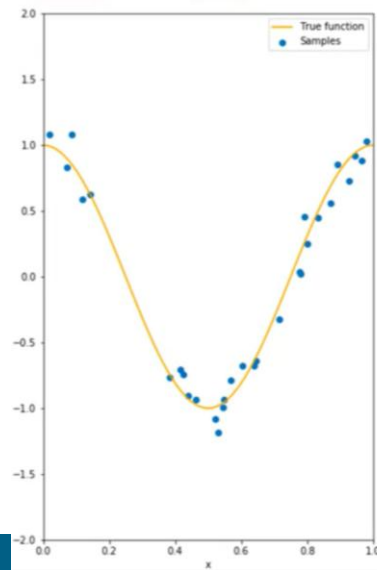


24



2.3. Chọn mô hình

$y(x) + \text{noise}$



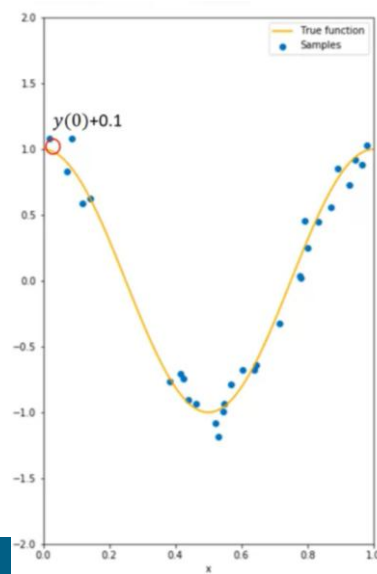
25

25



2.3. Chọn mô hình

$y(x) + \text{noise}$



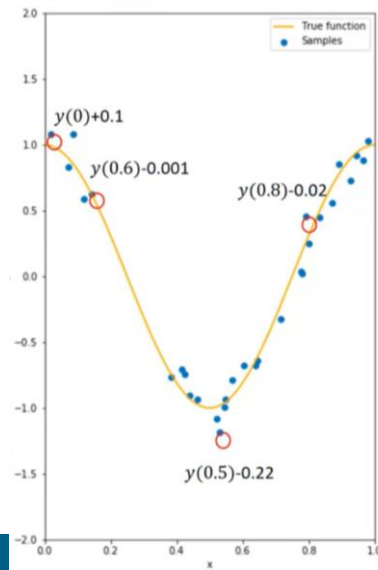
26

26



2.3. Chọn mô hình

$y(x) + \text{noise}$



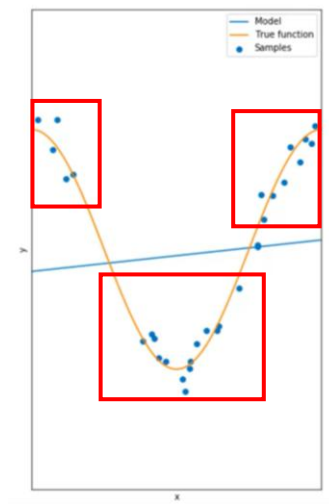
27

27



2.3. Chọn mô hình

$y = b_0 + b_1x$



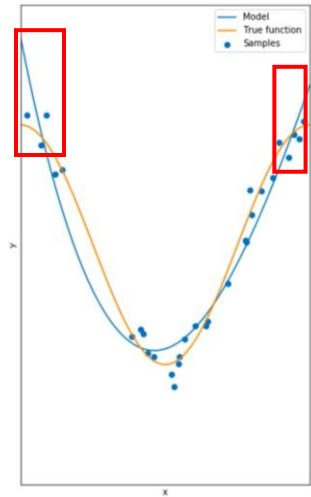
28

28



2.3. Chọn mô hình

$$y = b_0 + b_1x + b_2x^2$$



...

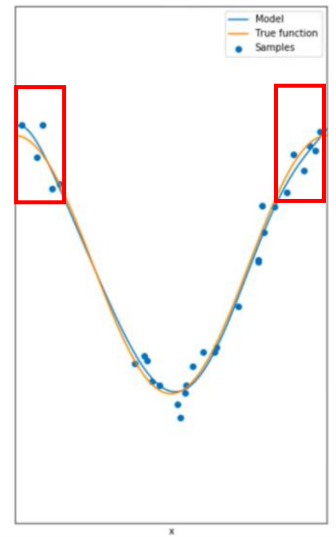
29

29



2.3. Chọn mô hình

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + b_4x^4 + b_5x^5 + b_6x^6 + b_7x^7 + b_8x^8$$



...

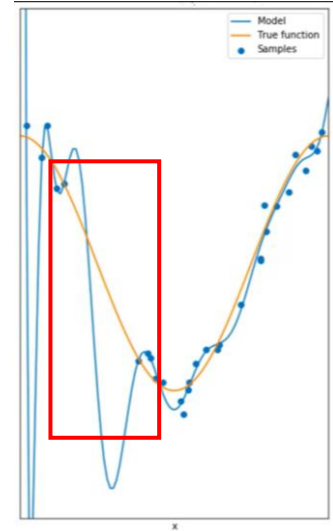
30

30



2.3. Chọn mô hình

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + b_4x^4 + b_5x^5 + b_6x^6 + b_7x^7 + b_8x^8 + \dots + b_{15}x^{15} + b_{16}x^{16}$$



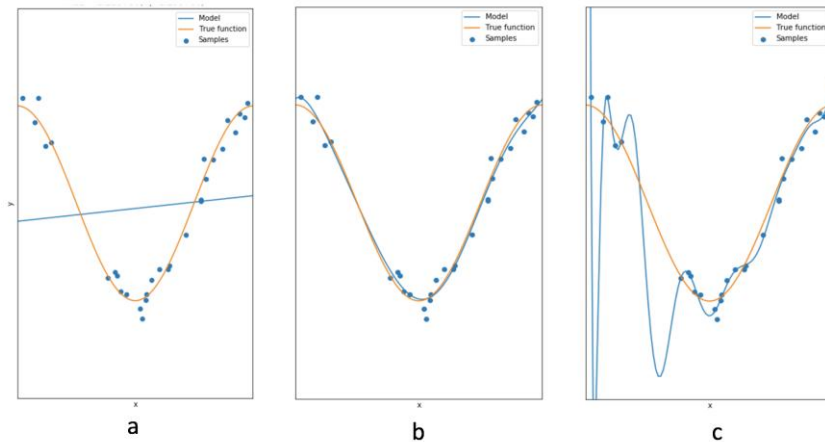
...

31

31



2.3. Chọn mô hình



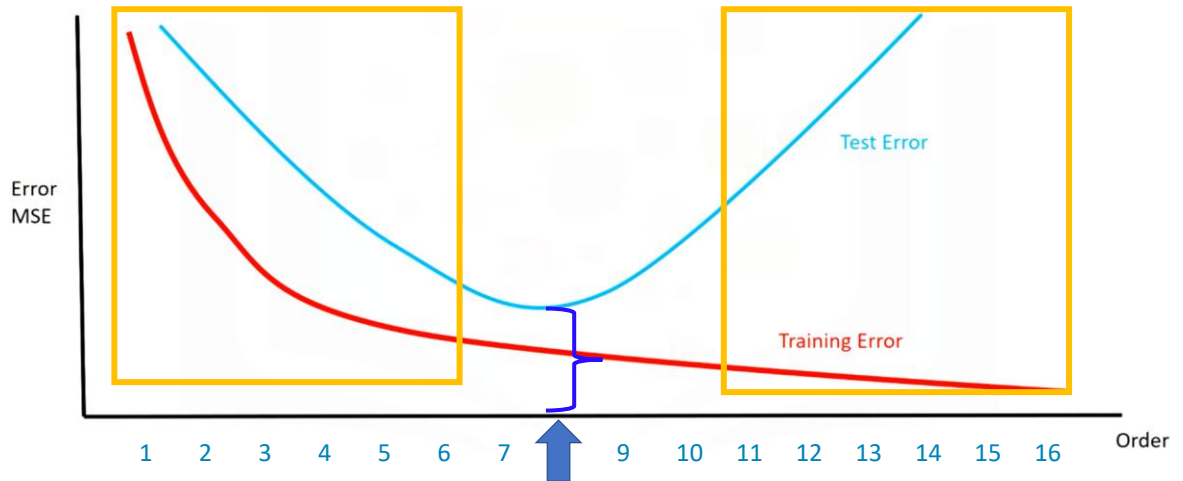
...

32

32



2.3. Chọn mô hình



33



2.4. Minh họa

Code

```
Rsqu_test=[]
order=[1,2,3,4]
for n in order:
    pr=PolynomialFeatures(degree=n)
    x_train_pr=pr.fit_transform(x_train[['horsepower']])
    x_test_pr=pr.fit_transform(x_test[['horsepower']])
    lr.fit(x_train_pr,y_train)
    Rsqu_test.append(lr.score(x_test_pr,y_test))
```

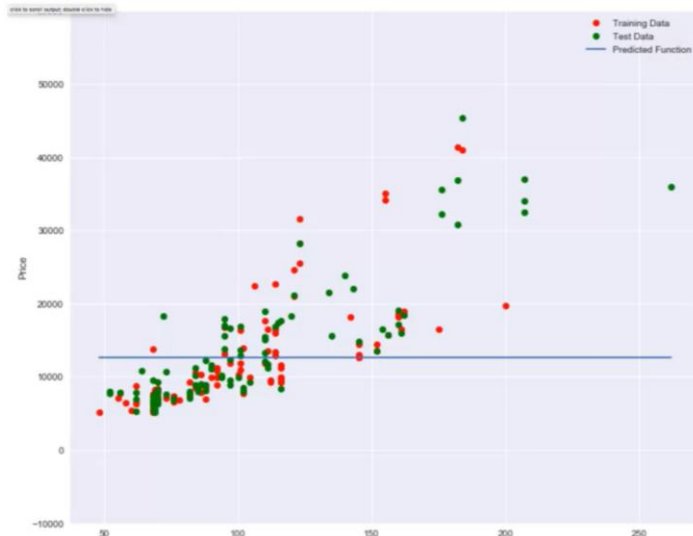
34

34



2.4. Minh họa

Chọn mô hình



35

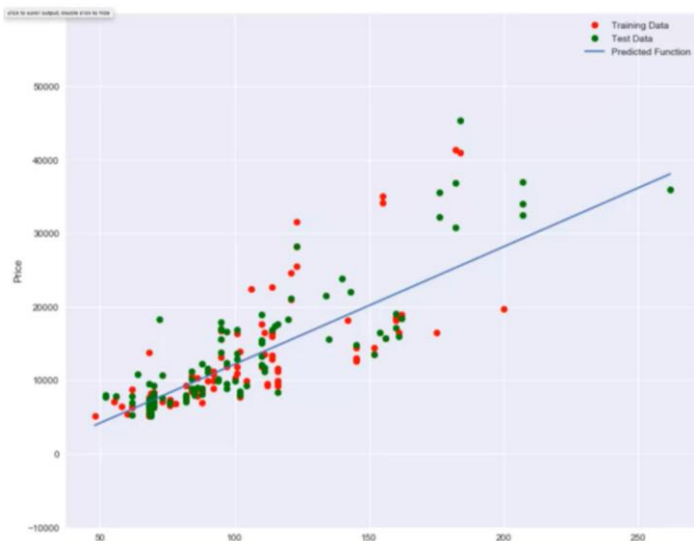
35



2.4. Minh họa

Chọn mô hình

Bậc $n = 1$



36

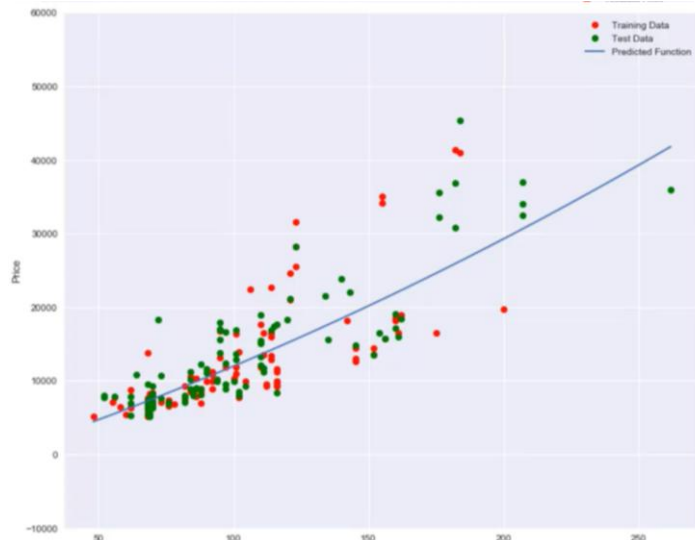
36



2.4. Minh họa

Chọn mô hình

Bậc $n = 2$



37

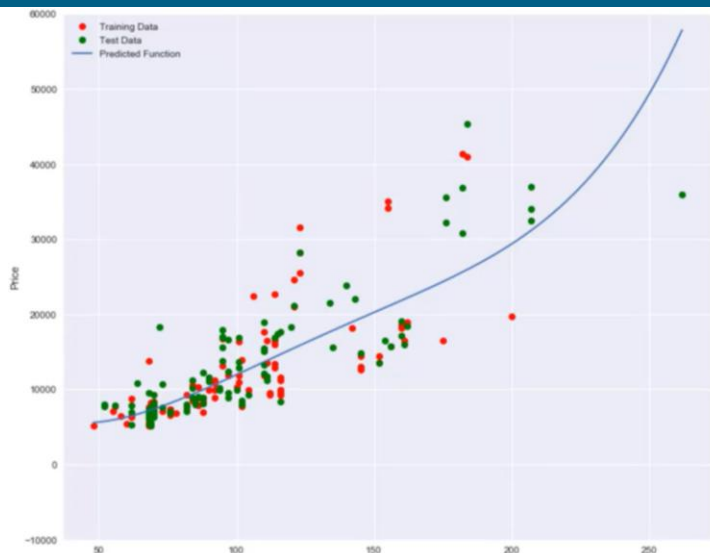
37



2.4. Minh họa

Chọn mô hình

Bậc $n = 3$



38

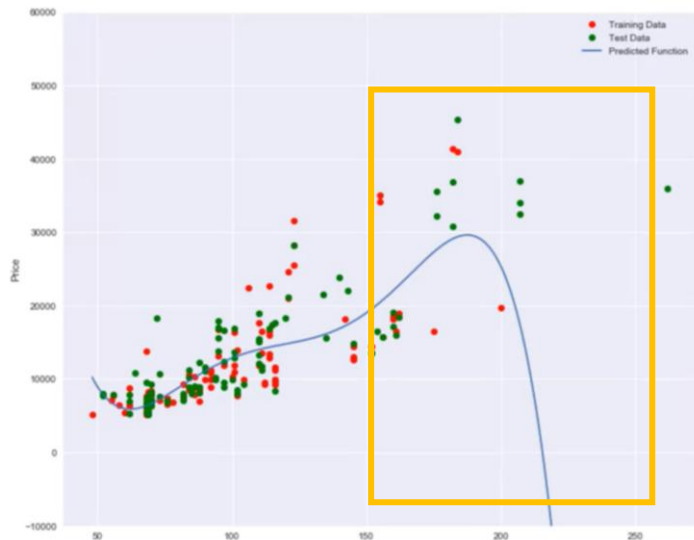
38



2.4. Minh họa

Chọn mô hình

Bậc $n = 4$



39

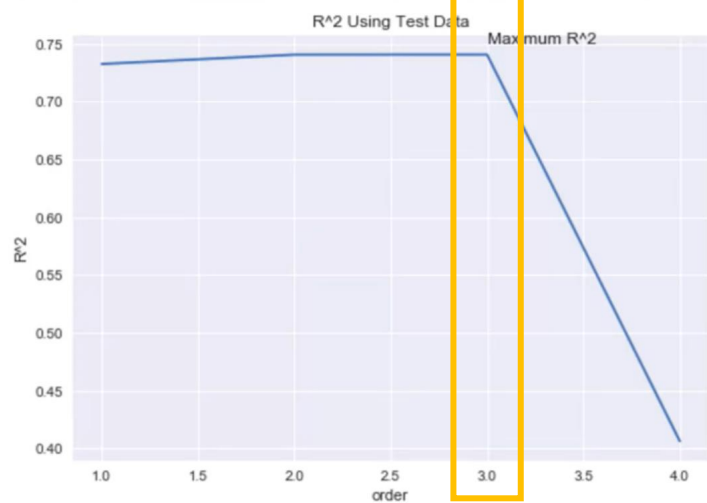
39



2.4. Minh họa

Chọn mô hình

```
plt.plot(order, Rsqu_test)
plt.xlabel('order')
plt.ylabel('R^2')
plt.title('R^2 Using Test Data')
plt.text(3, 0.75, 'Maximum R^2')
```



40

40

```

Rsqu_test = []

order = [1, 2, 3, 4]
for n in order:
    pr = PolynomialFeatures(degree=n)

    x_train_pr = pr.fit_transform(x_train[['horsepower']])
    x_test_pr = pr.fit_transform(x_test[['horsepower']])

    lr.fit(x_train_pr, y_train)

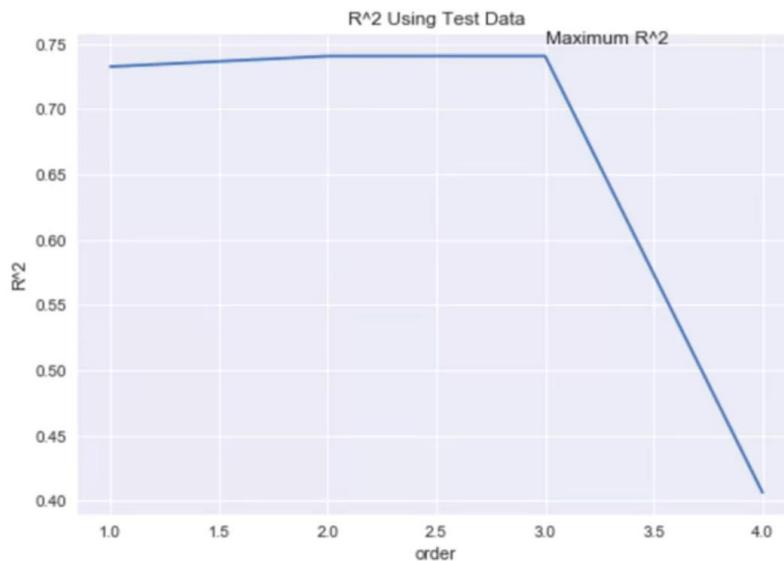
    Rsqu_test.append(lr.score(x_test_pr, y_test))

plt.plot(order, Rsqu_test)
plt.xlabel('order')
plt.ylabel('R^2')
plt.title('R^2 Using Test Data')
plt.text(3, 0.75, 'Maximum R^2 ')

```

41

41



42

42

```
Rsqu_test = []

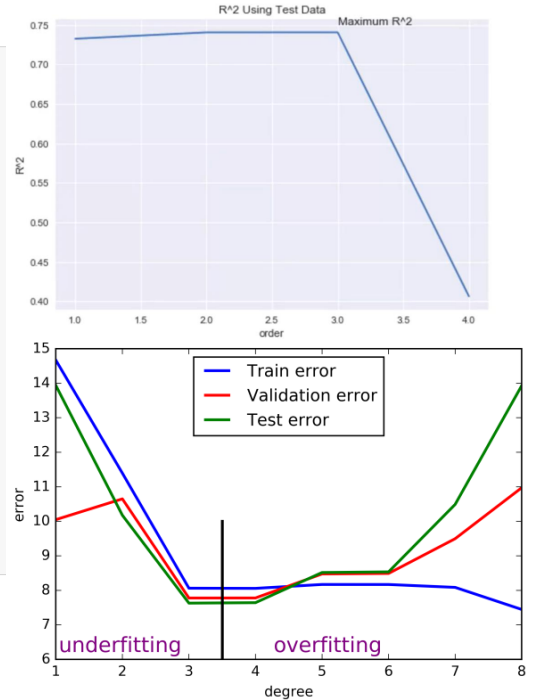
order = [1, 2, 3, 4]
for n in order:
    pr = PolynomialFeatures(degree=n)

    x_train_pr = pr.fit_transform(x_train[['horsepower']])
    x_test_pr = pr.fit_transform(x_test[['horsepower']])

    lr.fit(x_train_pr, y_train)

    Rsqu_test.append(lr.score(x_test_pr, y_test))

plt.plot(order, Rsqu_test)
plt.xlabel('order')
plt.ylabel('R^2')
plt.title('R^2 Using Test Data')
plt.text(3, 0.75, 'Maximum R^2')
```

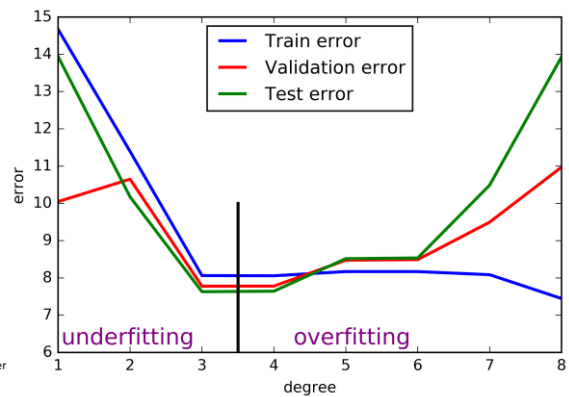
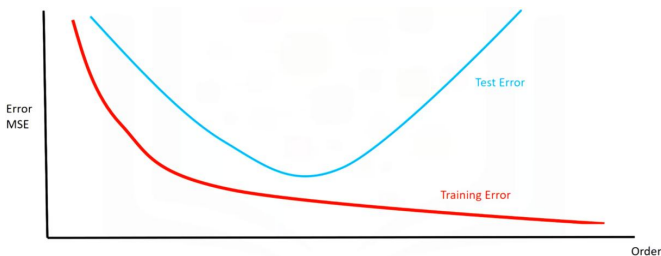


43

x_data = ['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']
y_data = ['price']

Cách 1: test_size=0.2

Cách 2
cv=5



44

44



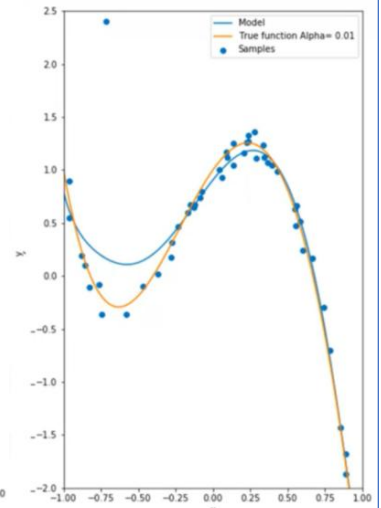
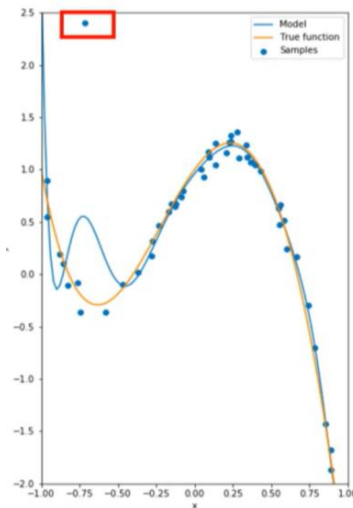
3. Hồi quy Ridge

Hồi quy Ridge:

- Hồi quy tuyến tính mở rộng
- Phân tích mối quan hệ giữa các biến độc lập và biến phụ thuộc.

Khi nào áp dụng hồi quy Ridge?

- Áp dụng khi giải quyết overfitting.
- Áp dụng khi bộ dữ liệu có các biến độc lập có mối liên hệ với nhau.



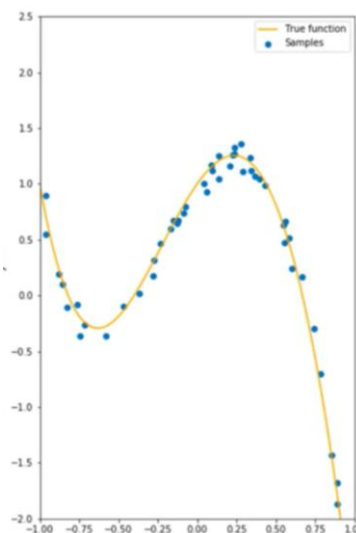
45

45



3. Hồi quy Ridge

$$y = 1 + 2x - 3x^2 - 4x^3 + x^4$$



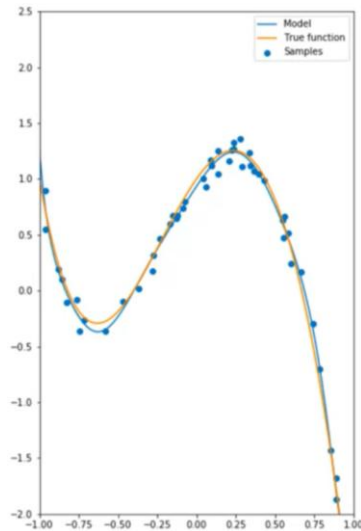
46

46



3. Hồi quy Ridge

$$y = 1 + 2x - 3x^2 - 4x^3 + x^4$$



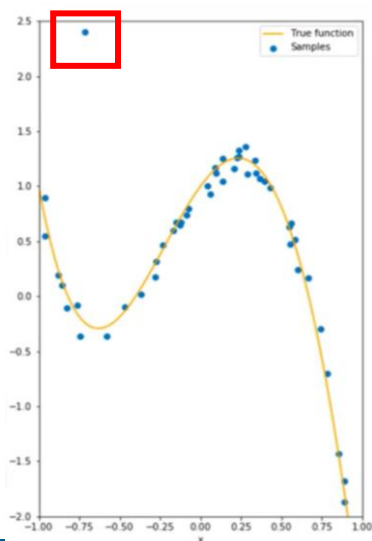
47

47



3. Hồi quy Ridge

$$1 + 2x - 3x^2 - 4x^3 + x^4$$



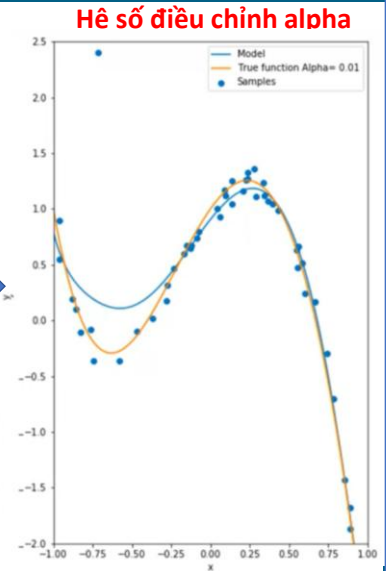
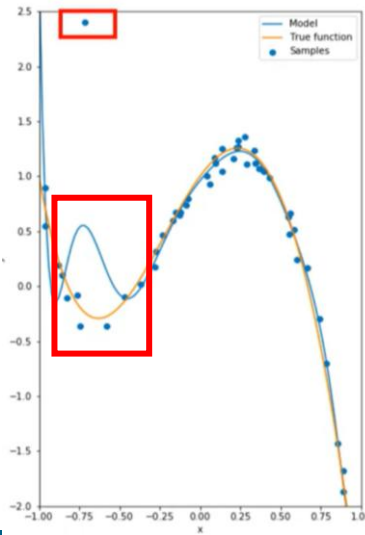
48

48



3. Hồi quy Ridge

$$1 + 2x - 3x^2 - 4x^3 + x^4$$



49

49



3. Hồi quy Ridge

Hệ số điều chỉnh alpha (α)

- Là một tham số điều chuẩn (regularization parameter)
- Được sử dụng để kiểm soát mức độ phạt lên các hệ số hồi quy của mô hình.
=> tức là, kiểm soát độ lớn của hệ số hồi quy: giảm thiểu sai số dự đoán và độ lớn của các hệ số hồi quy.
- Giảm hiện tượng overfitting

50

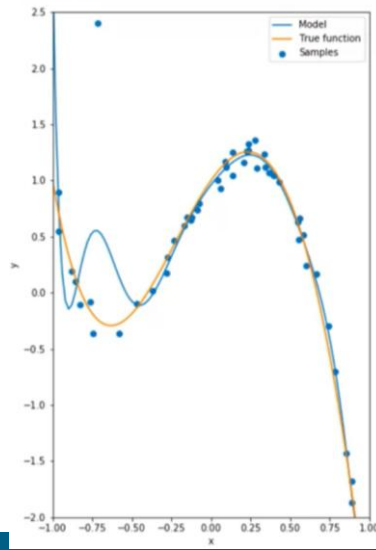
50



3. Hồi quy Ridge

Hệ số điều chỉnh **alpha**

Alpha
0
0.001
0.01
1
10



- $\alpha=0$, hồi quy Ridge là hồi quy tuyến tính thông thường.
- Không có thành phần điều chuẩn, và mô hình dễ bị quá khớp.

52

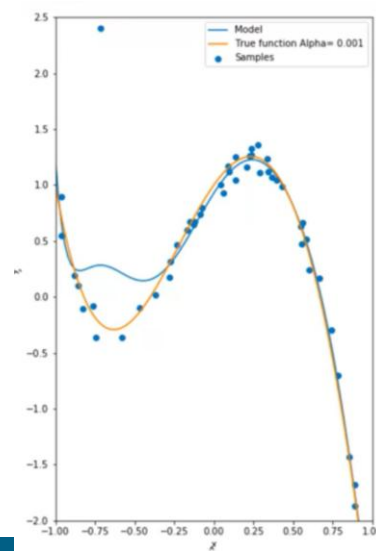
52



3. Hồi quy Ridge

Hệ số điều chỉnh **alpha**

Alpha
0
0.001
0.01
1
10



53

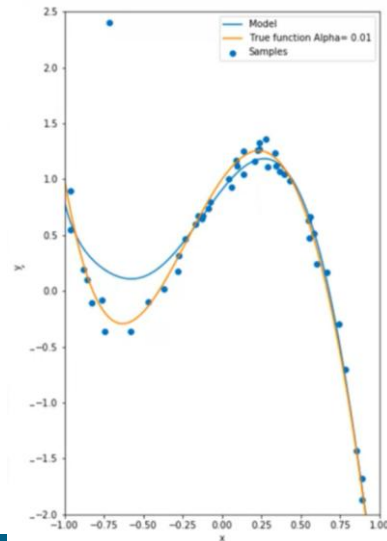
53



3. Hồi quy Ridge

Hệ số điều chỉnh **alpha**

Alpha
0
0.001
0.01
1
10



54

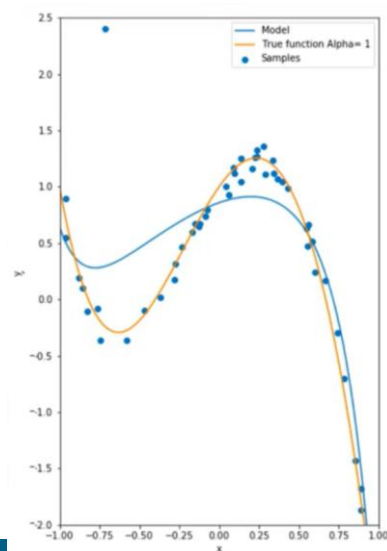
54



3. Hồi quy Ridge

Hệ số điều chỉnh **alpha**

Alpha
0
0.001
0.01
1
10



55

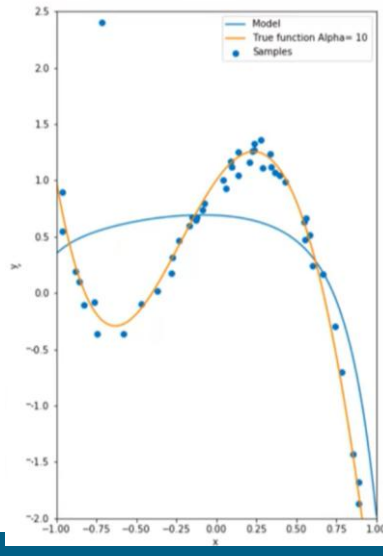
55



3. Hồi quy Ridge

Hệ số điều chỉnh **alpha**

Alpha
0
0.001
0.01
1
10



Vậy, dựa vào đâu để
xác định hệ số
điều chỉnh **alpha** là tốt nhất?

56

56



3. Hồi quy Ridge

Hệ số điều chỉnh alpha (α): Cách chọn α phù hợp

- Phụ thuộc vào **dữ liệu** cụ thể và đặc điểm của mô hình.
 - Nếu muốn thăm dò các mô hình thì chọn dãy rộng để quan sát.
Ví dụ, [0.001,0.01,0.1,1,10,100,1000] => có thể gọi là dãy leo thang
 - Dữ liệu ít nhiễu [0.0001,0.001,0.01,0.1]
 - Dữ liệu rất nhiễu [10,100,1000]
- Phương pháp tạo dãy leo: **`np.logspace(-3, 4, 60)`**
 - => $-3: 10^{-3} = 0.001$, $10^4 = 10_000$, số lượng giá trị
- Tiếp theo, kỹ thuật xác thực chéo (cross-validation) trên các tập dữ liệu khác nhau và chọn α mang lại hiệu suất tốt nhất trên tập kiểm tra.

57

57



3. Hồi quy Ridge

Cài đặt 1

```
from sklearn.linear_model import Ridge
RidgeModel=Ridge(alpha=0.1)
RidgeModel.fit(X,y)
Yhat=RidgeModel.predict(X)
```

...

59

59



3. Hồi quy Ridge

Cài đặt 2

...

60

60



3. Hồi quy Ridge

Cài đặt 2

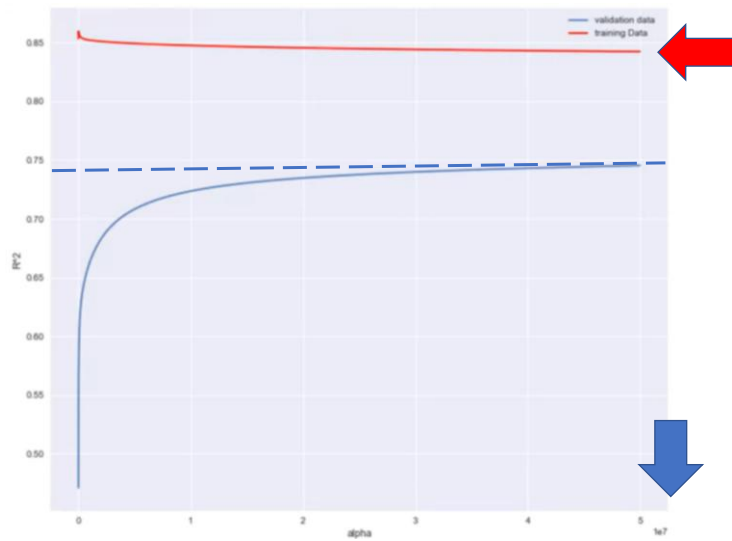
...

61

61



3. Hồi quy Ridge



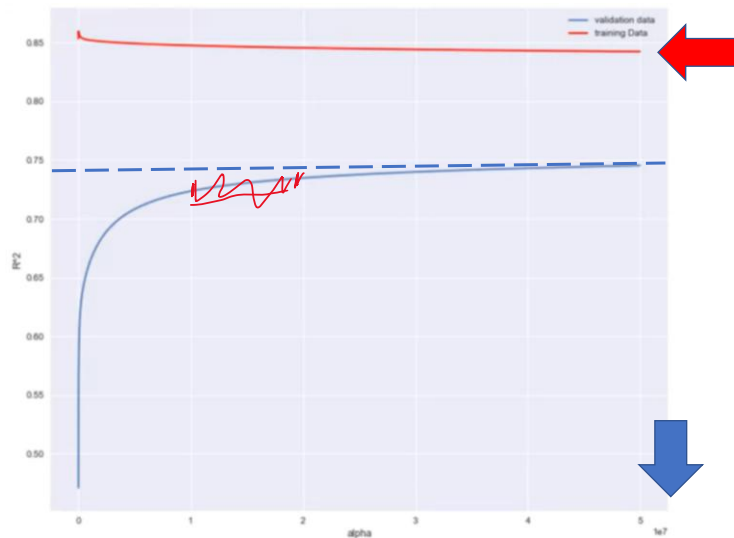
...

62

62



3. Hồi quy Ridge



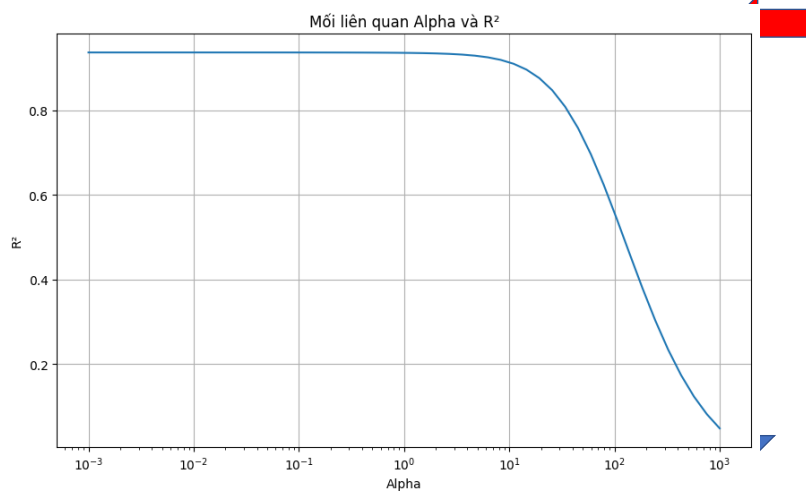
...

63

63



3. Hồi quy Ridge



...

64

64

```
x_data = ['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']  
y_data = ['price']
```

Phương pháp 1
test_size=0.2

Phương pháp 2
cv=5

65

65

```
x_data = ['horsepower', 'curb-weight', 'engine-size', 'highway-mpg']  
y_data = ['price']
```

Phương pháp 1
test_size=0.2

Phương pháp 2
cv=5

```
Rsqu_test = []  
order = [1, 2, 3, 4]  
for n in order:  
    pr = PolynomialFeatures(degree=n)  
  
    x_train_pr = pr.fit_transform(x_train[['horsepower']])  
    x_test_pr = pr.fit_transform(x_test[['horsepower']])  
  
    lr.fit(x_train_pr, y_train)  
  
    Rsqu_test.append(lr.score(x_test_pr, y_test))  
  
plt.plot(order, Rsqu_test)  
plt.xlabel('order')  
plt.ylabel('R^2')  
plt.title('R^2 Using Test Data')  
plt.text(3, 0.75, 'Maximum R^2 ')
```

66



68



Hỏi - Đáp



...

69

69